SPPU TE BE COMP Click to join telegram

---

1. Divide & conquer strategy

Divide & Conquer Strategy

Definition

Divide and Conquer is an algorithm design technique in which a problem is:

1. **Divided** into smaller sub-problems of the same type
2. **Conquered** by solving each sub-problem recursively
3. **Combined** to obtain the final solution

Steps of Divide & Conquer

1. **Divide**
   Split the problem into smaller independent sub-problems. 2.

Conquer
   Solve the sub-problems recursively.
   If the sub-problem is small enough, solve it directly.

3. **Combine**
   Merge the solutions of sub-problems to form the solution of the original problem.

General Structure (Pseudo-Code)

```
DivideConquer(problem)
{
    if (problem is small)
solve directly     else
        divide problem into subproblems
solve each subproblem recursively
combine the results
                                                                    }
```

Examples of Divide & Conquer Algorithms

Algorithm	Application
Binary Search	Searching

Algorithm	Application
Merge Sort	Sorting
Quick Sort	Sorting
Strassen's Matrix Multiplication	Matrix operations
Karatsuba Algorithm	Fast multiplication

Advantages

- Reduces problem complexity
- Efficient for large inputs
- Enables parallel processing
- Often improves time complexity

Disadvantages

- Recursive overhead
- Extra memory usage (e.g., Merge Sort)
- Not suitable for very small problems

Applications

- Searching and sorting
- Matrix multiplication
- Computational geometry
- Fast arithmetic algorithms

---

2. Merge sort.

Definition

Merge Sort is a **sorting algorithm** based on the **Divide & Conquer strategy**.

SPPU TE BE COMP Click to join telegram

It divides the array into smaller sub-arrays, sorts them, and then **merges** them to produce a sorted array.

Working Principle

Merge Sort works in **three steps**:

1. **Divide**
   Divide the array into two halves until each sub-array contains only one element. 2.
   Conquer
   Recursively sort the sub-arrays. 3.
   Combine (Merge)
   Merge the sorted sub-arrays to form a single sorted array.

Algorithm (Pseudo-Code)

```
MergeSort(arr, left, right)
{
    if (left < right)
    {
        mid = (left + right) / 2
MergeSort(arr, left, mid)
        MergeSort(arr, mid + 1, right)
        Merge(arr, left, mid, right)
    }
                                                                    }
```

Example

Input Array:

```
38 27 43 3 9 82 10
```

Divide Step

```
[38 27 43 3]   [9 82 10]
```

Merge Step

```
[3 27 38 43]   [9 10 82]
```

Final Sorted Array

```
                                                    3 9 10 27 38 43 82
```

Time Complexity

Case	Time
Best Case	O(n log n)
Average Case	O(n log n)
Worst Case	O(n log n)

Space Complexity

- **O(n)** (extra temporary array needed)

Advantages

✔ Stable sorting algorithm
✔ Efficient for large datasets
✔ Predictable time complexity

Disadvantages

✗ Requires extra memory
✗ Slower for small datasets due to recursion

Applications

- External sorting
- Sorting linked lists
- Large file sorting
- Used in stable sorting systems

SPPU TE BE COMP Click to join telegram

---

3. Quick sort.

Definition

Quick Sort is a **divide and conquer sorting algorithm** that works by selecting a **pivot element**, partitioning the array around the pivot, and recursively sorting the sub-arrays.

Working Principle

Quick Sort works in **three main steps**:

1. **Choose Pivot**
   Select an element as pivot (first, last, middle, or random).
2. **Partition**
   Rearrange the array so that: ○ Elements smaller than pivot are on the left ○ Elements greater than pivot are on the right 3. **Recursion**
   Apply Quick Sort to left and right sub-arrays.

Algorithm (Pseudo-Code)

```
QuickSort(arr, low, high)
{     if (low <
high)
    {
        p = Partition(arr, low, high)
        QuickSort(arr, low, p - 1)
        QuickSort(arr, p + 1, high)
    }
                                                                    }
```

Example

Input Array:

```
10 7 8 9 1 5
```

Pivot (last element): 5

After Partition

```
1 | 5 | 10 7 8 9
```

Final Sorted Array

```
1 5 7 8 9 10
```

Time Complexity

Case	Time
Best Case	O(n log n)
Average Case	O(n log n)
Worst Case	O(n²)

Space Complexity

- **O(log n)** (recursive stack)

Advantages

✔ Fast in practice
✔ In-place sorting
✔ No extra memory needed

Disadvantages

✗ Worst case O(n²)
✗ Not a stable sort

Applications

- General-purpose sorting
- Large datasets
- Used in system libraries

SPPU TE BE COMP Click to join telegram

4. Matrix multiplication

Definition

Matrix multiplication is an operation in which two matrices are multiplied to produce a third matrix, following specific rules of rows and columns.

If

- Matrix **A** is of order **m** × **n**
- Matrix **B** is of order **n** × **p**

Then the resulting matrix **C** will be of order **m** × **p**.

Condition for Matrix Multiplication

Matrix multiplication is possible **only if**:

$$\text{Number of columns of A} = \text{Number of rows of B}$$

Formula

$$C[i][j] = \sum_{k=0}^{n-1} A[i][k] \times B[k][j]$$

Algorithm (Standard Method)

```
for i = 0 to m-1      for
j = 0 to p-1
C[i][j] = 0          for
k = 0 to n-1
            C[i][j] = C[i][j] + A[i][k] * B[k][j]
```

Example

Let

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \quad B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$$

Result

$$C = \begin{bmatrix} (1 \times 5 + 2 \times 7) & (1 \times 6 + 2 \times 8) \\ (3 \times 5 + 4 \times 7) & (3 \times 6 + 4 \times 8) \end{bmatrix} = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$$

Time Complexity

- **$O(n^3)$** for standard matrix multiplication

Space Complexity

- **$O(n^2)$** for result matrix

Advantages

✔ Simple and easy to implement
✔ Works for all compatible matrices

Disadvantages

✗ Slow for large matrices
✗ High computational cost

Applications

- Graphics and image processing
- Machine learning and AI

SPPU TE BE COMP Click to join telegram

- Scientific simulations
- Computer graphics (3D transformations)

---

5. Greedy strategy.

Definition

Greedy Strategy is an algorithm design technique that **makes the best (optimal) choice at each step**, with the hope that these local optimal choices lead to a **global optimal solution**.

Characteristics of Greedy Algorithms

1. Makes **locally optimal decisions**
2. Does **not reconsider** previous choices
3. Simple and fast
4. Works correctly only for certain problems

Steps of Greedy Strategy

1. **Selection**
   Choose the best option available at the current step.
2. **Feasibility Check**
   Check whether the chosen option satisfies problem constraints. 3.

Solution Construction
Add the choice to the solution set. 4.

Termination
Repeat until the solution is complete.

General Structure (Pseudo-Code)

```
Greedy(problem)
{
    solution = empty
    while (solution not complete)
    {
```

```
        choose best option
if (choice is feasible)
add choice to solution     }
    return solution
                                                                    }
```

Examples of Greedy Algorithms

Algorithm	Application
Activity Selection	Scheduling
Fractional Knapsack	Resource allocation
Prim’s Algorithm	Minimum Spanning Tree
Kruskal’s Algorithm	Minimum Spanning Tree
Dijkstra’s Algorithm	Shortest path

Advantages

- ✔ Easy to implement
- ✔ Fast execution
- ✔ Low time complexity

Disadvantages

- ✗ Does not always give optimal solution
- ✗ Not suitable for all problems
- ✗ Decision is irreversible

Applications

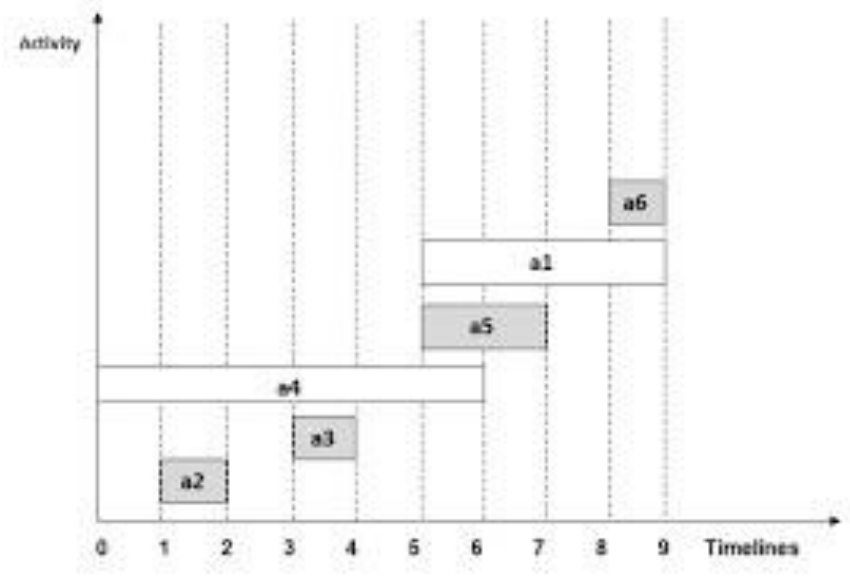
- Network routing
- Scheduling problems
- Resource allocation
- Graph algorithms

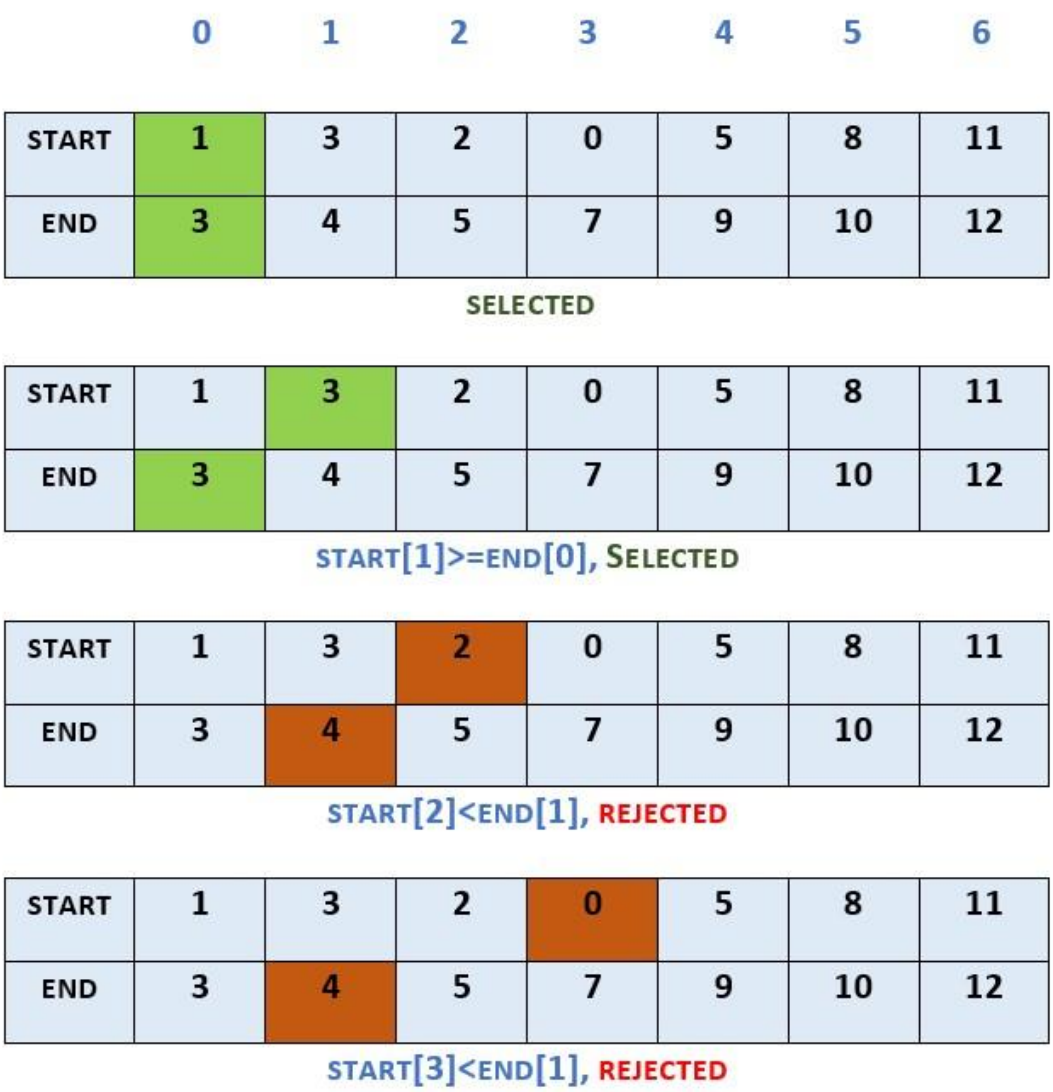
SPPU TE BE COMP Click to join telegram

---

6. Activity selection.

Activity Selection Problem





Definition

The **Activity Selection problem** is a **greedy algorithm problem** where the goal is to **select the maximum number of non-overlapping activities** that can be performed by a **single person or resource**, given their **start and finish times**.

Problem Statement

Given:

- A set of activities
- Each activity has:
  - **Start time (s_i)** ○ **Finish time (f_i)**

SPPU TE BE COMP Click to join telegram

Select the **maximum number of activities** such that **no two activities overlap**.

Greedy Choice

Always **select the activity that finishes earliest**.
This leaves maximum time for the remaining activities.

Algorithm (Steps)

1. Sort activities in **ascending order of finish time**
2. Select the **first activity** 3. For each next activity:
    - If its start time ≥ finish time of last selected activity
    - Select it

Pseudo-Code

```
ActivitySelection(s, f, n)
{
    select activity 1
last = 1     for i =
2 to n
    {
        if (s[i] >= f[last])
        {
            select activity i
last = i
        }
    }
                                                                    }
```

Example

Activity	Start	Finish
A1	1	2
A2	3	4
A3	0	6

A4	5	7
A5	8	9
A6	5	9

Selected Activities

A1 → A2 → A4 → A5

Time Complexity

- **O(n log n)** (due to sorting)

Space Complexity

- **O(1)** (excluding input)

Advantages

✔ Simple greedy solution
✔ Efficient
✔ Optimal result

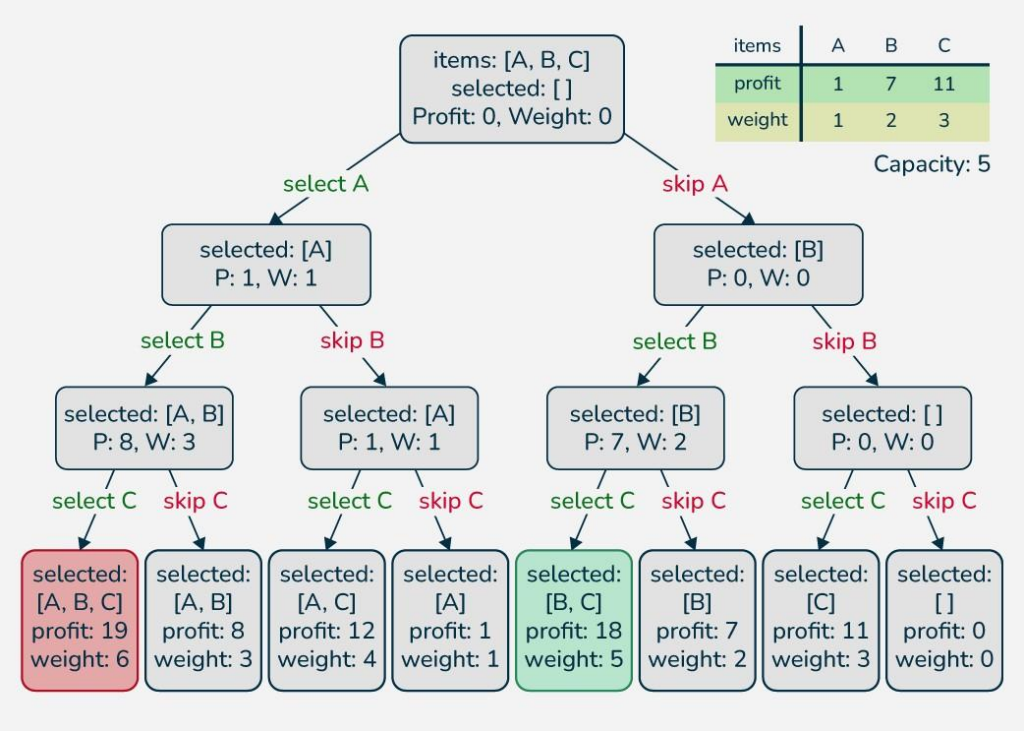
Applications

- CPU scheduling
- Classroom or meeting room scheduling
- Resource allocation
- Job scheduling

---

7. Fractional knapsack

SPPU TE BE COMP Click to join telegram



Definition

The **Fractional Knapsack problem** is a **greedy algorithm problem** where items can be **broken into fractions**.
The objective is to **maximize total profit** without exceeding the **knapsack capacity**.

Problem Statement

Given:

- **n items**
- Each item has: ○ **Weight (w_i)**
  - **Profit (p_i)**
- **Knapsack capacity = W**

Choose items (or fractions of items) such that:

$$\text{Total weight} \leq W$$

and **profit is maximum**.

Greedy Choice

Select items in **decreasing order of profit/weight ratio**:

$$\text{Ratio} = \frac{p_i}{w_i}$$

Algorithm (Steps)

1. Calculate **profit/weight ratio** for each item
2. Sort items in **descending order of ratio**
3. Add items fully if possible
4. If capacity is insufficient, add **fraction of the item**

Pseudo-Code

```
FractionalKnapsack(W, items)
{
    sort items by (profit/weight) in descending order
totalProfit = 0

    for each item
    {
        if item.weight <= W
        {
            W = W - item.weight
totalProfit += item.profit
        }
else          {
            fraction = W / item.weight
totalProfit += item.profit * fraction
break
        }
}
    return totalProfit
                                                                    }
```

Example

Item	Weight	Profit	p/w
A	10	60	6
B	20	100	5
C	30	120	4

Knapsack capacity W = 50

SPPU TE BE COMP Click to join telegram

Selection

- Take **A** (10, profit 60)
- Take **B** (20, profit 100)
- Take **2/3 of C** (20, profit 80)

Maximum Profit

60+100+80=240

Time Complexity

- **O(n log n)** (sorting)

Space Complexity

- **O(1)**

Advantages

✔ Gives optimal solution
✔ Simple greedy approach
✔ Efficient

Disadvantages

✗ Not applicable when items cannot be divided
✗ Different from 0/1 Knapsack

Applications

- Cargo loading
- Resource allocation
- Budget management
- Cutting and filling problems

8. Huffman coding.

Definition

Huffman Coding is a **greedy compression algorithm** used to **encode characters with variable-length binary codes**, where **frequently occurring characters get shorter codes** and **less frequent characters get longer codes**.

Objective

- Reduce total number of bits used to represent data
- Achieve **lossless data compression**

Working Principle

1. Count **frequency** of each character
2. Create a **leaf node** for each character
3. Insert all nodes into a **min-priority queue** (based on frequency) 4. Repeat until one node remains:
    - Remove two nodes with **smallest frequency** ○ Create a new internal node with frequency = sum ○ Insert it back into the queue
5. Assign:
    - **0** to left edge ○ **1** to right edge
6. Read codes from **root to leaf**

Algorithm (Steps)

```
HuffmanCoding(characters, frequencies)
{
    create min-priority queue Q
insert all characters as nodes into Q
     while (Q.size >
1)
    {
        x = extractMin(Q)
y = extractMin(Q)
        z = new node with freq = x.freq + y.freq
```

```
        z.left = x
        z.right = y
insert z into Q
    }
    return root of Huffman tree
                                                                    }
```

Example

Character	Frequency
A	5
B	9
C	12
D	13
E	16
F	45

Possible Huffman Codes

Character	Code
F	0
C	100
D	101
A	1100
B	1101
E	111

(Exact codes may vary, but total length is minimum.)

Properties

- Prefix-free codes (no code is prefix of another)
- Optimal variable-length coding
- Greedy choice property

Time Complexity

- **O(n log n)** (priority queue operations)

Advantages

✔ Optimal compression
✔ Lossless encoding
✔ Widely used

Disadvantages

✗ Requires storing the tree
✗ Less efficient for very small data

Applications

- File compression (ZIP, GZIP)
- Image compression (PNG)
- Audio/video compression
- Data transmission

---

9. Huffman tree construction

Definition

Huffman Tree construction is the process of building a **binary tree** used in **Huffman Coding**, where:

- Each **leaf node** represents a character
- The **weight** of each node is the character's frequency
- Characters with **lower frequency** are placed deeper in the tree

---

Steps to Construct a Huffman Tree

1. Create a **leaf node** for each character with its frequency

2. Insert all nodes into a **min-priority queue**
3. While more than one node exists in the queue:
    - Remove two nodes with **minimum frequency** ○ Create a new internal node with:

      frequency=f1+f2

    - Make the two removed nodes its left and right children ○ Insert the new node back into the queue
4. The **remaining node** becomes the **root of the Huffman Tree**

Example

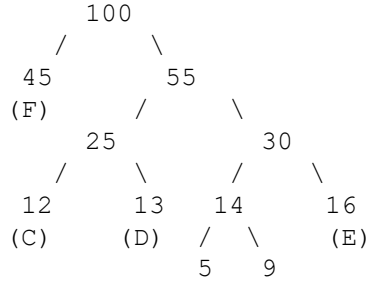
Given Characters and Frequencies

Character	Frequency
A	5
B	9
C	12
D	13
E	16
F	45

Step-by-Step Construction

1. Pick **A(5)** and **B(9)** → create node **14**
2. Pick **C(12)** and **D(13)** → create node **25**
3. Pick **14** and **E(16)** → create node **30**
4. Pick **25** and **30** → create node **55**
5. Pick **F(45)** and **55** → create node **100 (root)**

Final Huffman Tree (Conceptual)



(A) (B)

Code Assignment Rule

- Left edge → **0**
- Right edge → **1**

(Exact codes may differ, but **total cost is minimum**.)

Time Complexity

- **O(n log n)**

Important Properties

- Binary tree
- Prefix-free encoding
- Optimal compression
- Greedy approach

---

10. Dynamic programming definition

Dynamic Programming (DP)

Definition

Dynamic Programming is an algorithm design technique used to solve complex problems by **breaking them into overlapping subproblems**, solving each subproblem **only once**, and **storing their results** to avoid repeated computation.

SPPU TE BE COMP Click to join telegram

Key Idea

Solve once, store the result, and reuse it whenever needed.

Characteristics of Dynamic Programming

1. **Optimal Substructure**
   The optimal solution of a problem can be constructed from optimal solutions of its subproblems.
2. **Overlapping Subproblems**
   The same subproblems are solved multiple times in a naive recursive approach.

Approaches in Dynamic Programming

1. Top-Down Approach (Memoization)

- Uses recursion
- Stores results in a table (array or map)

2. Bottom-Up Approach (Tabulation)

- Builds solution iteratively
- Fills DP table from base cases

General Structure (Pseudo-Code)

```
DP(problem) {
initialize dp table
for each subproblem
        dp[i] = best solution using previous results
return dp[final]
                                                    }
```

Examples of Dynamic Programming Problems

Problem	Application
Fibonacci series	Basic DP
0/1 Knapsack	Resource optimization
Longest Common Subsequence	String matching
Matrix Chain Multiplication	Optimization

Shortest paths (Bellman-Ford)	Graphs

Advantages

- ✔ Reduces time complexity
- ✔ Avoids redundant calculations
- ✔ Efficient for optimization problems

Disadvantages

- ✗ Uses extra memory
- ✗ Hard to identify DP problems initially

---

11. DP vs greedy

Feature	Dynamic Programming	Greedy
Decision making	Considers **all possibilities**	Chooses **best immediate option**
Optimality	Always gives **optimal solution**	Optimal only for some problems
Subproblems	Overlapping subproblems	No subproblem storage
Memory usage	High (DP table)	Very low
Time complexity	Usually higher	Usually lower
Backtracking	Possible	Not possible
Implementation	Complex	Simple

Problem	Technique Used

SPPU TE BE COMP Click to join telegram

0/1 Knapsack	Dynamic Programming
Fractional Knapsack	Greedy
Longest Common Subsequence	Dynamic Programming
Activity Selection	Greedy
Matrix Chain Multiplication	Dynamic Programming
Problem	**Technique Used**
Huffman Coding	Greedy

---

12. 0/1 knapsack DP.

Definition

The **0/1 Knapsack problem** is an **optimization problem** where each item can be either:

- **Included (1)** completely, or
- **Excluded (0)**

Fractions of items are **not allowed**.

The objective is to **maximize total profit** without exceeding the **knapsack capacity**.

Problem Statement

Given:

- **n items**
- Each item has: ○ Weight w_i ○ Profit p_i
- Knapsack capacity W

Find the **maximum profit** such that:

$$\sum w_i \leq W$$

Why Dynamic Programming?

- Greedy approach **fails** for 0/1 Knapsack
- Problem has:
  - **Optimal substructure**
  - **Overlapping subproblems**

Hence, **Dynamic Programming** is used

DP Table Definition

Let:

$$dp[i][w] = \text{Maximum profit using first } i \text{ items with capacity } w$$

Recurrence Relation

If the item **cannot be included:**

$$dp[i][w] = dp[i-1][w] \quad \text{if } w_i > w$$

If the item **can be included:**

$$dp[i][w] = \max(dp[i-1][w],\ p_i + dp[i-1][w-w_i])$$

Algorithm (Pseudo-Code)

```
for i = 0 to n
    for w = 0 to W
        if i == 0 or w == 0             dp[i][w] =
0         else if w[i] <= w             dp[i][w] =
max(dp[i-1][w],                           p[i] +
dp[i-1][w-w[i]])          else
            dp[i][w] = dp[i-1][w]
```

SPPU TE BE COMP Click to join telegram

Example

Item	Weight	Profit
1	1	1
2	3	4
Item	Weight	Profit
3	4	5
4	5	7

Knapsack Capacity W = 7

Result

- Maximum Profit = **9**
- Items selected: **Item 2 and Item 3**

Time Complexity

- **O(nW)**

Space Complexity

- **O(nW)**
  (can be optimized to O(W))

Advantages

✔ Always gives optimal solution
✔ Systematic and reliable

Disadvantages

✗ High memory usage
✗ Slower for large capacity values

Applications

- Resource allocation
- Budget planning
- Project selection
- Cargo loading

---

13. LCS algorithm.

Definition

The **Longest Common Subsequence (LCS)** problem finds the **longest sequence of characters** that appears in the **same relative order** (but not necessarily contiguously) in **two strings**.

Problem Statement

Given two strings:

- **X** of length m
- **Y** of length n

Find the **length** (and optionally the sequence) of their **Longest Common Subsequence**.

Why Dynamic Programming?

- Problem has **optimal substructure**
- Subproblems **overlap**
- Recursive solution is inefficient

SPPU TE BE COMP Click to join telegram

DP Table Definition

Let:

$$dp[i][j] = \text{Length of LCS of } X[0..i-1] \text{ and } Y[0..j-1]$$

Recurrence Relation

Case 1: Characters Match

$$\text{If } X[i-1] = Y[j-1]$$

$$dp[i][j] = 1 + dp[i-1][j-1]$$

Case 2: Characters Do Not Match

$$\text{If } X[i-1] \neq Y[j-1]$$

$$dp[i][j] = \max(dp[i-1][j],\ dp[i][j-1])$$

Algorithm (Pseudo-Code)

```
LCS(X, Y) {     m =
length(X)     n =
length(Y)     create
dp[m+1][n+1]
     for i = 0 to m
dp[i][0] = 0     for
j = 0 to n
        dp[0][j] = 0

    for i = 1 to m         for j = 1 to n
            if X[i-1] == Y[j-1]
dp[i][j] = 1 + dp[i-1][j-1]
else
                dp[i][j] = max(dp[i-1][j], dp[i][j-1])

    return dp[m][n]
}
```

Example

Let:

- **X = "AGGTAB"**
- **Y = "GXTXAYB"**

LCS

```
GTAB
```

Length of LCS

```
4
```

Time Complexity

- **O(mn)**

Space Complexity

- **O(mn)**
  (can be optimized to O(min(m, n)))

Applications

- File comparison (diff tools)
- DNA sequence analysis
- Version control systems
- Spell checking

---

14. Floyd-Warshall algorithm

Definition

Floyd–Warshall algorithm is a **dynamic programming algorithm** used to find the **shortest paths between all pairs of vertices** in a **weighted graph**.

- Works for **directed and undirected graphs**

SPPU TE BE COMP Click to join telegram

- Can handle **negative edge weights**
- Does **not** work with negative cycles

Key Idea

For every pair of vertices `(i, j)`, check whether a **path via an intermediate vertex `k`** is shorter than the current known path.

Algorithm Concept

Algorithm Concept

Let:

$$dist[i][j] = \text{shortest distance from } i \text{ to } j$$

Update rule:

$$dist[i][j] = \min(dist[i][j],\ dist[i][k] + dist[k][j])$$

Algorithm (Pseudo-Code)

```
FloydWarshall(graph, V)
{
    dist = graph
     for k = 1 to V
for i = 1 to V
for j = 1 to V
                if dist[i][k] + dist[k][j] < dist[i][j]
dist[i][j] = dist[i][k] + dist[k][j] }
```

Example

Initial distance matrix:

$$\begin{matrix} 0 & 3 & \infty & 7 \\ 8 & 0 & 2 & \infty \\ 5 & \infty & 0 & 1 \\ 2 & \infty & \infty & 0 \end{matrix}$$

After applying Floyd–Warshall, we get shortest distances between **all pairs of vertices.**

Time Complexity

- **$O(V^3)$**

Space Complexity

- **$O(V^2)$**

Advantages

✔ Computes all-pairs shortest paths
✔ Simple DP approach
✔ Handles negative edges

Disadvantages

✗ High time complexity
✗ Not suitable for very large graphs

Applications

- Network routing
- Traffic systems
- Graph analysis
- Distance matrix computation

---

15. Applications of divide & conquer.

SPPU TE BE COMP Click to join telegram

Applications of Divide & Conquer

1. Sorting Algorithms

- **Merge Sort • Quick Sort**
- Divide the array into subarrays, sort them, and combine results.

---

2. Searching Algorithms

- **Binary Search**
- Reduces search space by half in each step.

---

3. Matrix Multiplication

- **Strassen's Matrix Multiplication**
- Reduces time complexity compared to standard method.

---

4. Computational Geometry

- Closest pair of points problem
- Convex hull algorithms

---

5. Fast Arithmetic Algorithms

- **Karatsuba multiplication**
- Fast exponentiation

---

6. Tree and Graph Algorithms

- Divide trees or graphs into smaller components
- Used in traversal and analysis

---

7. Big Integer Computations

- Multiplication of very large numbers
- Used in cryptography

8. Image Processing

- Image segmentation
- Fast Fourier Transform (FFT)

Advantages in Applications

✔ Reduces problem complexity
✔ Efficient for large inputs
✔ Supports parallel processing

---

16. Applications of greedy.

Applications of Greedy Strategy

1. Activity Selection Problem

- Selects the maximum number of non-overlapping activities.
- Used in **scheduling** problems.

---

2. Fractional Knapsack Problem

- Maximizes profit by selecting items or fractions of items.
- Used in **resource allocation**.

---

3. Huffman Coding

- Generates optimal prefix codes for data compression.
- Used in **file and data compression**.

SPPU TE BE COMP 👈 Click to join telegram

4. Minimum Spanning Tree (MST)

- **Prim's Algorithm**
- **Kruskal's Algorithm**
- Used in **network design** (roads, cables, pipelines).

---

5. Shortest Path Algorithms

- **Dijkstra's Algorithm**
- Finds shortest path from a source vertex.
- Used in **GPS and routing systems**.

---

6. Job Sequencing with Deadlines

- Schedules jobs to maximize profit within deadlines.
- Used in **CPU scheduling**.

---

7. Optimal Merge Pattern

- Minimizes cost of merging files.
- Used in **external sorting**.

---

8. Coin Change (Greedy Version)

- Finds minimum number of coins (for standard denominations).
- Used in **ATM machines**.

Advantages in Applications

✔ Fast execution
✔ Simple implementation
✔ Efficient for real-time systems

17. DP applications.

Applications of Dynamic Programming

1. Knapsack Problems

- **0/1 Knapsack**
- **Unbounded Knapsack**
- Used in **resource allocation** and **budget planning**.

---

2. Shortest Path Algorithms

- **Floyd–Warshall Algorithm**
- **Bellman–Ford Algorithm**
- Used in **network routing** and **navigation systems**.

---

3. String Matching Problems

- **Longest Common Subsequence (LCS)**
- **Edit Distance**
- Used in **file comparison**, **DNA analysis**, and **spell checking**.

---

4. Matrix Chain Multiplication

- Finds the most efficient way to multiply matrices.
- Used in **database query optimization**.

---

5. Fibonacci and Sequence Problems

- Fibonacci series
- Tribonacci, Catalan numbers

---

6. Optimal Binary Search Tree

- Minimizes average search cost.

SPPU TE BE COMP 👈 Click to join telegram

- Used in **database indexing**.

---

7. Coin Change Problem

- Finds minimum number of coins for a given amount.
- Used in **financial systems**.

---

8. Game Theory Problems

- Optimal game strategies
- Used in **AI and game development**.

---

9. Scheduling Problems

- Weighted interval scheduling
- Job allocation problems

Advantages in Applications

✔ Guarantees optimal solution
✔ Avoids redundant computation
✔ Efficient for optimization problems

18. Optimal substructure.

Optimal Substructure

Definition

A problem is said to have **optimal substructure** if an **optimal solution of the problem can be constructed from optimal solutions of its subproblems**.

Key Idea

Solve subproblems optimally → combine them → get overall optimal solution

This property is essential for:

- **Dynamic Programming**
- Many **Greedy algorithms**

Explanation

If solving a large problem requires solving smaller parts of the same problem, and each part's **best solution contributes to the best overall solution**, then the problem has optimal substructure.

Example 1: Shortest Path

- The shortest path from **A to C** through **B** consists of:
  - Shortest path from **A to B** ○ Shortest path from **B to C**

Example 2: 0/1 Knapsack

- The optimal selection of items depends on the optimal solution of remaining capacity and items.

Example 3: Longest Common Subsequence (LCS)

- LCS of two strings is built using LCS of their prefixes.

Algorithms Using Optimal Substructure

Algorithm	Technique
0/1 Knapsack	Dynamic Programming
LCS	Dynamic Programming
Floyd–Warshall	Dynamic Programming
Bellman–Ford	Dynamic Programming
Dijkstra	Greedy + Optimal Substructure

Difference from Overlapping Subproblems

Feature	Optimal Substructure	Overlapping Subproblems
Meaning	Solution built from sub-solutions	Same subproblems repeat
Required for DP	Yes	Yes

19. Overlapping subproblems

Overlapping Subproblems

Definition

A problem is said to have **overlapping subproblems** when it can be broken into **smaller subproblems that are solved repeatedly** during a recursive solution.

Key Idea

Same subproblems occur again and again

Dynamic Programming avoids this repetition by **storing results** and **reusing them**.

Explanation

In many recursive algorithms, the same function calls are made multiple times with the same inputs.
This causes **wasteful recomputation**.

Dynamic Programming improves efficiency by:

- **Memoization** (top-down)
- **Tabulation** (bottom-up)

Example 1: Fibonacci Series

Recursive Approach

$$F(n) = F(n-1) + F(n-2)$$

Here, values like `F(3)` and `F(2)` are computed **multiple times**.

DP Approach

Store results of `F(n)` so each value is computed **only once**.

Example 2: Longest Common Subsequence (LCS)

- LCS of prefixes of strings is computed many times.
- DP table stores each result once.

Example 3: 0/1 Knapsack

- Same capacity and item combinations repeat.
- DP table avoids recomputation.

Algorithms with Overlapping Subproblems

Algorithm	Technique
Fibonacci	Dynamic Programming
LCS	Dynamic Programming
0/1 Knapsack	Dynamic Programming
Matrix Chain Multiplication	Dynamic Programming
Floyd–Warshall	Dynamic Programming

Difference from Optimal Substructure

Feature	Overlapping Subproblems	Optimal Substructure
Meaning	Same subproblems repeat	Solution built from sub-solutions
DP Requirement	Yes	Yes

20. Memoization vs tabulation.

Memoization	Tabulation
Top-down approach	Bottom-up approach
Uses recursion	Uses iteration
Stores results when needed	Fills DP table step by step
Recursive function calls	No recursion
Extra memory for call stack	No call stack overhead
Risk of stack overflow	No stack overflow
Easier to implement	More efficient in practice

NUMERICALS

Perform Merge Sort on {32,5,12,7,25,19}.

Perform Merge Sort

Given Array

```
32, 5, 12, 7, 25, 19
```

Step 1: Divide (Split)

```
[32, 5, 12]        [7, 25, 19]
```

Further divide:

```
[32] [5, 12]       [7] [25, 19]
```

Again divide:

```
[32] [5] [12]      [7] [25] [19]
```

Step 2: Conquer & Merge

Merge single elements

```
[5] + [12]  →  [5, 12]
[25] + [19] →  [19, 25]
```

Merge subarrays

```
[32] + [5, 12]   →  [5, 12, 32]
```

```
[7] + [19, 25]   →  [7, 19, 25]
```

Step 3: Final Merge

```
[5, 12, 32] + [7, 19, 25]
```

Comparison order:

```
5 < 7  → 5
7 < 12 → 7
12 < 19 → 12
19 < 32 → 19
25 < 32 → 25
```

```
Remaining → 32
```

☑ Final Sorted Array

```
5, 7, 12, 19, 25, 32
```

Perform Quick Sort on {40,10,70,20,50,30}.

Perform Quick Sort

Given Array

```
40, 10, 70, 20, 50, 30
```

Step 1: First Partition

Pivot = 30

Rearrange elements:

```
10, 20 | 30 | 40, 70, 50
```

Array after partition:

```
10, 20, 30, 40, 70, 50
```

Step 2: Left Sub-array (Before Pivot)

Sub-array:

```
10, 20
```

Pivot = 20

After partition:

```
10 | 20
```

Left part is sorted.

Step 3: Right Sub-array (After Pivot)

Sub-array:

```
40, 70, 50
```

Pivot = 50

After partition:

```
40 | 50 | 70
```

Right part is sorted.

☑ Final Sorted Array

```
10, 20, 30, 40, 50, 70
```

Solve fractional knapsack (given values & weights).

Fractional Knapsack (Solved Example)

Given (Chosen Data)

Item	Weight (w)	Profit (p)
A	10	60

B	20	100
C	30	120

Knapsack Capacity (W) = 50

Step 1: Compute Profit / Weight Ratio

Item	Weight	Profit	p / w
A	10	60	6
B	20	100	5
C	30	120	4

Step 2: Sort Items (Descending p/w)

Order after sorting:

`A → B → C`

Step 3: Fill the Knapsack

1️⃣ Take Item A (fully)

- Weight used = 10
- Profit = 60
- Remaining capacity = 50 − 10 = 40

2️⃣ Take Item B (fully)

- Weight used = 20
- Profit = 100
- Remaining capacity = 40 − 20 = 20

☐ Take Item C (fraction)

- Available capacity = 20
- Fraction taken = 20 / 30 = 2/3
- Profit from C = 120 × (2/3) = 80

☑ Final Result

Total Profit

60+100+80=24060 + 100 + 80 = \boxed{240}60+100+80=240

Items Selected

- Item A → 100%
- Item B → 100%
- Item C → 2/3

Construct Huffman coding tree for given frequencies.

Construct Huffman Coding Tree

(Given Frequencies – Chosen Example)

Given Characters and Frequencies

Character	Frequency
A	5
B	9

Character	Frequency
C	12
D	13
E	16
F	45

Step 1: Arrange in Ascending Order of Frequency

`A(5), B(9), C(12), D(13), E(16), F(45)`

Step 2: Combine Two Minimum Frequencies Repeatedly

◇ Step 1

- Combine **A(5)** and **B(9)**
- New node = **14**

`(A, B) → 14`

◇ Step 2

- Combine **C(12)** and **D(13)**
- New node = **25**

`(C, D) → 25`

◇ Step 3

- Combine **14** and **E(16)**
- New node = **30**

`((A,B), E) → 30`

◇ Step 4

- Combine **25** and **30**

- New node = **55**

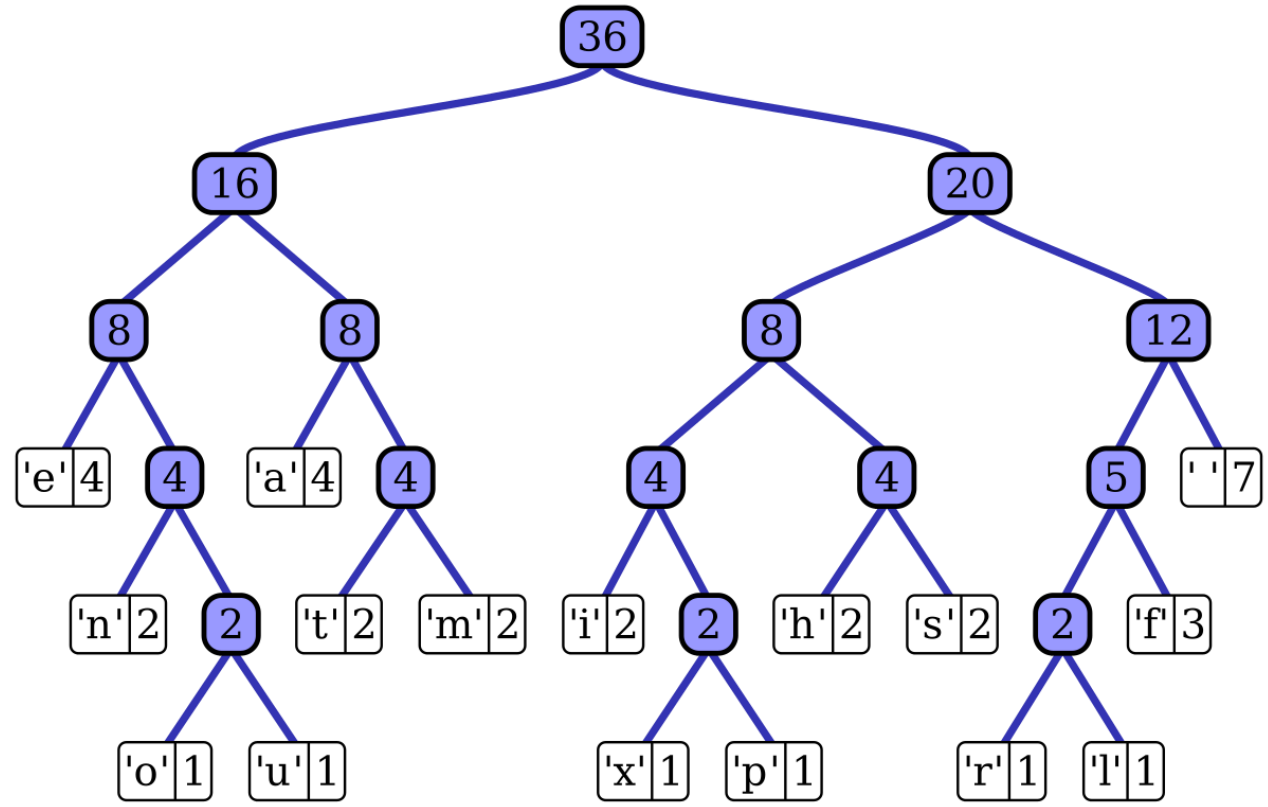
```
((C,D), (A,B,E)) → 55
```

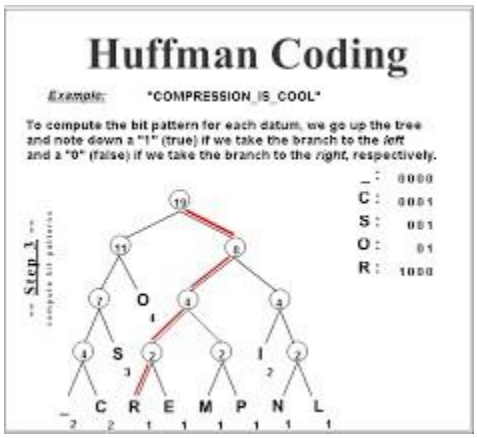
◇ Step 5

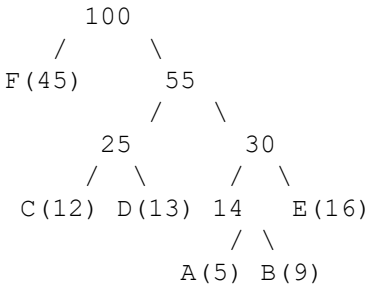
- Combine **F(45)** and **55**
- New node = **100 (Root)**

```
(F, rest) → 100
```

Step 3: Final Huffman Tree







Step 4: Assign Codes

(Rule: **Left = 0**, **Right = 1**)

Character	Huffman Code
F	0
C	100
D	101
A	1100
B	1101
E	111

(Codes may differ slightly, but total cost will always be minimum.)

Solve 0/1 knapsack using DP for 4 items.

Solve 0/1 Knapsack using DP (4 Items)

Given (Chosen Example)

Item	Weight (w)	Profit (p)
1	1	1
Item	**Weight (w)**	**Profit (p)**
2	3	4
3	4	5
4	5	7

Knapsack Capacity (W) = 7

Step 1: DP Table Definition

$$dp[i][w] = \text{Maximum profit using first } i \text{ items and capacity } w$$

- Rows → Items (0 to 4)
- Columns → Capacity (0 to 7)

Step 2: DP Recurrence

- If $w_i > w$:

$$dp[i][w] = dp[i-1][w]$$

- Else:

$$dp[i][w] = \max(dp[i-1][w],\ p_i + dp[i-1][w-w_i])$$

Step 3: DP Table Construction

i \ W	0	1	2	3	4	5	6	7
0	0	0	0	0	0	0	0	0

1	0	1	1	1	1	1	1	1
2	0	1	1	4	5	5	5	5
3	0	1	1	4	5	6	6	9
4	0	1	1	4	5	7	8	9

Step 4: Final Answer

- **Maximum Profit** = **9**
- **Knapsack Capacity Used** = 7
- **Items Selected** = **Item 2 (w=3, p=4)** and **Item 3 (w=4, p=5)**

Time & Space Complexity

- **Time Complexity:** O(nW)
- **Space Complexity:** O(nW)

Apply Floyd–Warshall on given 3x3 adjacency matrix.

Given Adjacency Matrix

Let the vertices be 1, 2, 3.

$$D^{(0)} = \begin{bmatrix} 0 & 3 & \infty \\ \infty & 0 & 1 \\ 2 & \infty & 0 \end{bmatrix}$$

Floyd–Warshall Formula

$$D[i][j] = \min(D[i][j],\ D[i][k] + D[k][j])$$

Step 1: Using Vertex 1 as Intermediate (k = 1)

Check all pairs (i, j) through vertex 1.

- No shorter paths are found.

$$D^{(1)} = \begin{bmatrix} 0 & 3 & \infty \\ \infty & 0 & 1 \\ 2 & \infty & 0 \end{bmatrix}$$

Step 2: Using Vertex 2 as Intermediate (k = 2)

- Update (1 → 3):
  $3 + 1 = 4 < \infty$ ✅

$$D^{(2)} = \begin{bmatrix} 0 & 3 & 4 \\ \infty & 0 & 1 \\ 2 & \infty & 0 \end{bmatrix}$$

Step 3: Using Vertex 3 as Intermediate (k = 3)

- Update (2 → 1):
  $1 + 2 = 3 < \infty$ ✅

$$D^{(3)} = \begin{bmatrix} 0 & 3 & 4 \\ 3 & 0 & 1 \\ 2 & \infty & 0 \end{bmatrix}$$

✅ Final Shortest Path Matrix

$$\boxed{\begin{bmatrix} 0 & 3 & 4 \\ 3 & 0 & 1 \\ 2 & \infty & 0 \end{bmatrix}}$$